National University of Computer & Emerging Sciences

Lecture 15

Scheduling III

Schedulers

- Short term Scheduler or CPU Scheduling
  - Which program is to be run next
- Long term Scheduler or Job Scheduler
  - Which ready jobs should be brought to memory
  - May need to invoke only when a process leaves the system
  - Must make a careful selection

Non Preemptive / Preemptive Scheduling

- Non Preemptive Scheduling
  - Once the CPU has been allocated to a process, it keeps it until it Terminates or has to wait for:
    - I/O
    - Child process etc.

- Preemptive Scheduling
  - Possible to get the CPU back from a process, even if the process has not completed its execution.

Scheduling Algorithms

- First come, First serve
- Shortest Job First
- Priority Scheduling
- Round-Robin Scheduling
- CFS
- Multi-level Queue Scheduling
- Multi-level Feed back queue Scheduling

An overview of CFS

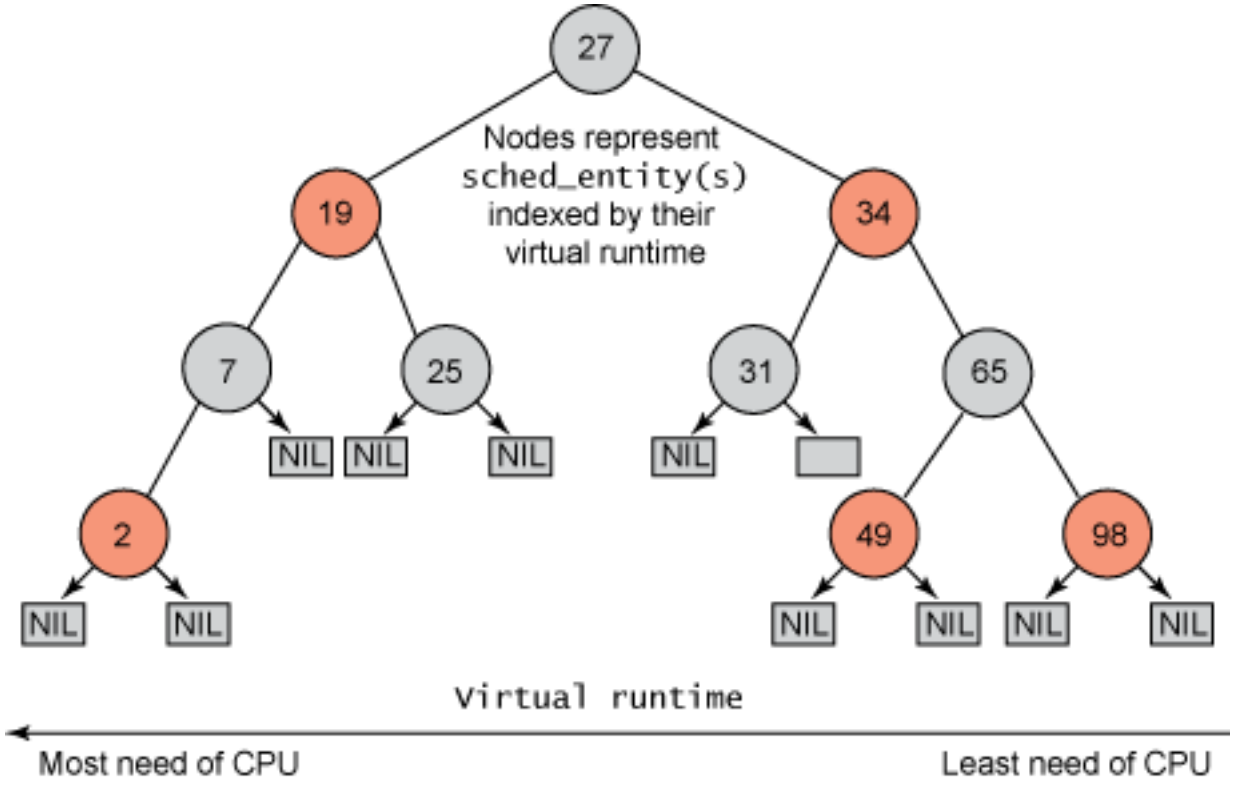
Completely Fair Scheduler

- The main idea behind the CFS is to maintain balance (fairness) in providing processor time to tasks.

- To determine the balance, the CFS maintains the amount of time provided to a given task in what's called the *virtual runtime.*
  - The smaller a task's virtual runtime the higher its need for the processor.

An overview of CFS

- Rather than maintaining the tasks in a run queue, as has been done in prior Linux schedulers, the CFS maintains a time-ordered red-black tree.


27
Nodes represent
sched_entity(s)
indexed by their
virtual runtime
19
34
7
25
31
65
NIL
NIL
NIL
NIL
NIL
2
49
98
NIL
NIL
NIL
NIL
NIL
NIL
Virtual runtime
Most need of CPU
Least need of CPU

An overview of CFS

- The scheduler then, to be fair, picks the left-most node of the red-black tree to schedule next to maintain fairness.

- The task accounts for its time with the CPU by adding its execution time to the virtual runtime and is then inserted back into the tree if runnable.

An overview of CFS

- In this way, tasks on the left side of the tree are given time to execute, and the contents of the tree migrate from the right to the left to maintain fairness.

Multilevel Queue Scheduling

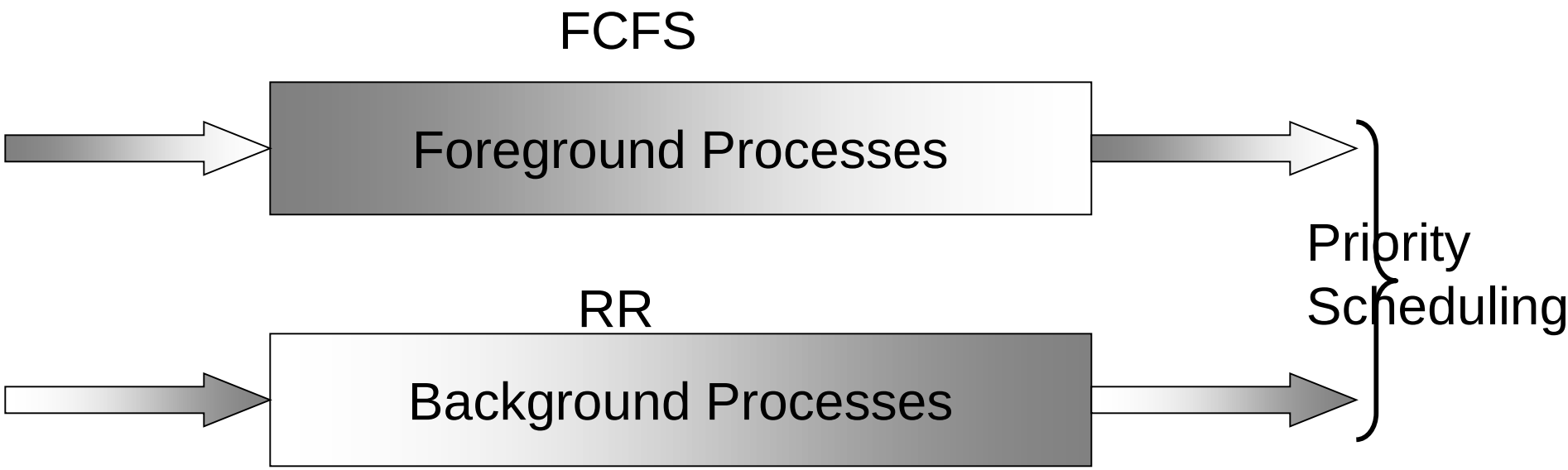
- Sometimes processes are classified into groups
- One classification can be:
  - Foreground (or Interactive) processes
  - Background (or batch) processes
- Different response time requirement
- => Different scheduling requirements
- Foreground processes usually have higher priorities

Multilevel Queue Scheduling (MQS)

- Partition the Ready queue into a number of queues
- Processes are permanently assigned to one of the queues
- Each queue may have its own scheduling algorithm
- In addition, there must be scheduling between the queues

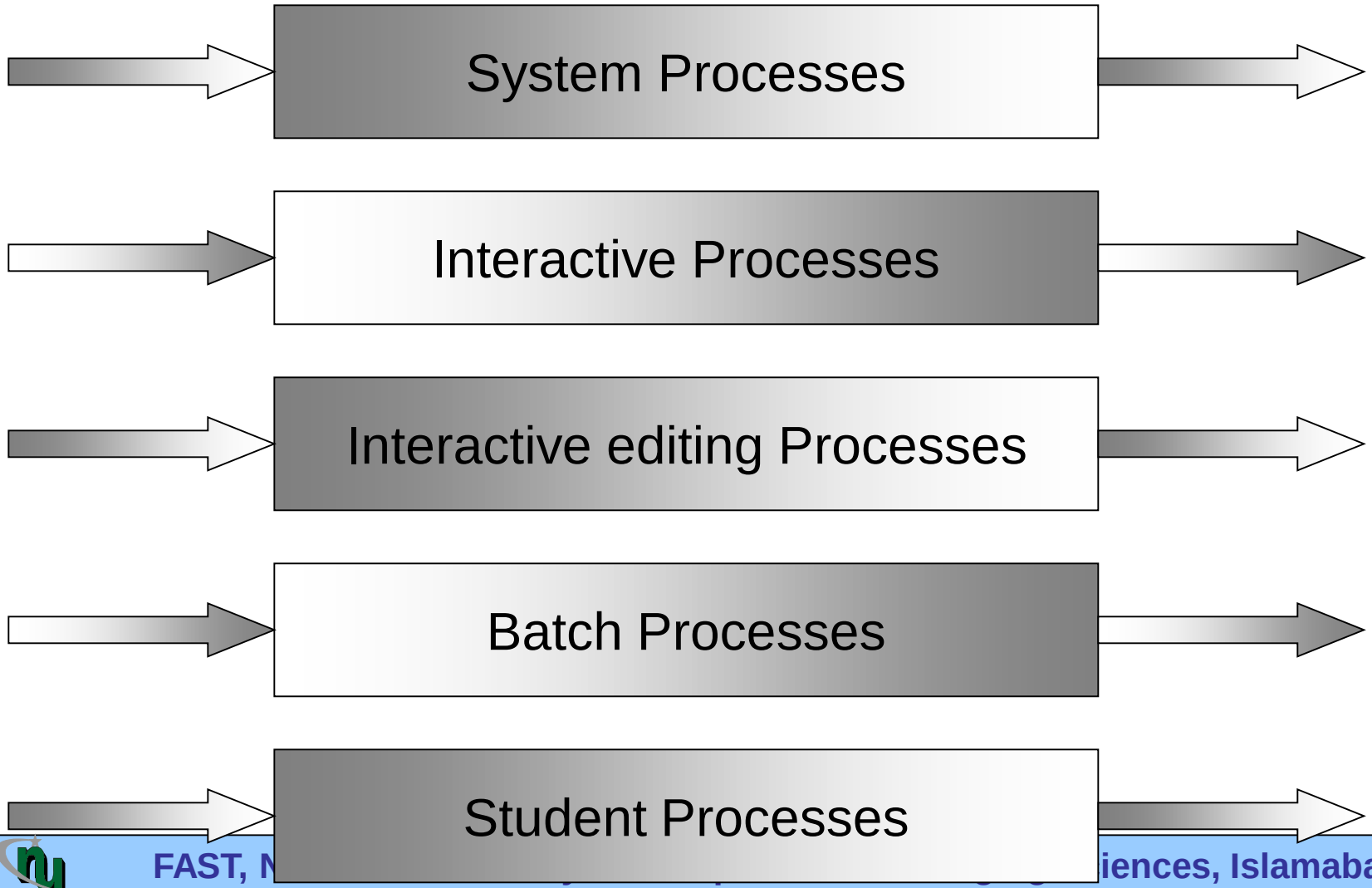
Multilevel Queue Scheduling

- Example:



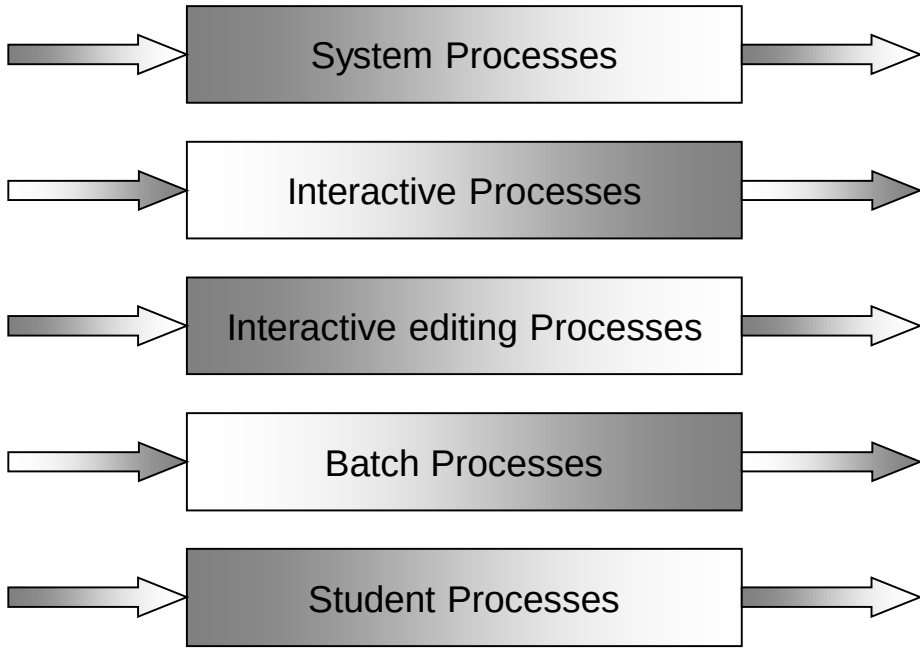
Multilevel Queue Scheduling

- Example:



Multilevel Queue Scheduling

- Each queue may have absolute priority over the other queue
- Alternatively, Time slice between the queues
  - Time slots can be equal
  - Or
    - 80% time for Foreground processes
    - 20% time for Background processes



Multilevel Feedback Queue Scheduling

- In Multilevel Queue a process is permanently assigned to a queue
- The queue to which a process should belong is decided statically
- Multilevel Feedback Queue Scheduling:
  - A Process may move between the Queues
  - Aging can be implemented this way.

Multilevel Feedback Queue Scheduling

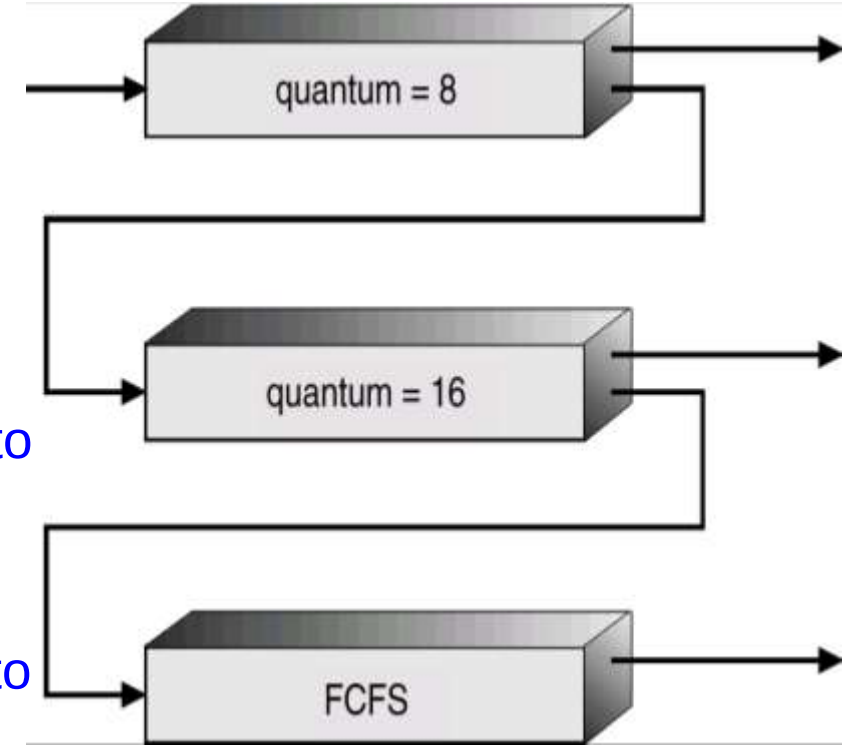
- Multilevel-feedback-queue scheduler defined by:
  - Number of queues
  - Scheduling algorithms for each queue
  - Method used to select when upgrade process
  - Method used to select when demote process
  - Method used to determine which queue a process will enter when that process needs service

Multilevel Feedback Queue Scheduling

- Example
  - If a process used too much CPU time, then move it to a lower-priority queue
  - If a process waits too long in a lower priority queue, then move it to a higher priority queue

Multilevel Feedback Queue Scheduling

- Example: Three queues:
  - Q_0 – RR time quantum 8 milliseconds
  - Q_1 – RR time quantum 16 milliseconds
  - Q_2 – FCFS
- Scheduling
  - A new job enters queue Q_0 served by RR.
  - Then job receives 8 milliseconds.
  - If not finished in 8 milliseconds, moved to Q_1.
  - At Q_1 job served by RR.
  - Then receives 16 milliseconds.
  - If not complete, preempted and moved to Q_2.



MLFQ - Working

- A new job enters queue Q1 which is served RR
  - When it gains CPU, job receives 8 milliseconds
  - If it does not finish in 8 milliseconds, job is moved to queue Q2

- At Q2 job is again served FCFS/RR and receives 16 additional milliseconds
  - If it still does not complete, it is preempted and moved to queue Q3

- If a process does not use up its quantum in the current level, it will keep its current queuing level and be put into the end of the queue. Then, it can still get the same amount of quantum (not remaining quantum) next time when it is picked.

Example

Process	*Arrival* Time	Burst Time
P_1	0	36
P_2	16	20
P_3	20	12

- The Gantt chart is:

